new Rails method provided for system tests—to set the Capybara driver to rack_test.

With that done, let's run the tests:

```
$ docker-compose exec web rspec spec/system/
.

Finished in 27.2 seconds (files took 11.57 seconds to load)
1 example, 0 failures
```

Great! We've configured Capybara and our system specs.

Running Tests That Rely on JavaScript

OK, let's take this up a notch. Setting up testing with JavaScript support is not going to be so straightforward with Docker. But I think you're ready to handle it.

Imagine, if you will, that we have an enhanced version of our /welcome page that has extra behavior that only works with JavaScript enabled. In fact, when functioning correctly, this JavaScript literally adds the message "ENHANCED!" on the page.

Here's my rather crude implementation in app/views/welcome/index.html.erb:

```
<% content_for :head do %>
  <script type="text/javascript">
    document.addEventListener("DOMContentLoaded",function(){
      document.getElementsByTagName('h1')[0].append(' ENHANCED!');
    });
  </script>
<% end %>

<h1>This page has been viewed <%= pluralize(@page_hits, 'time') %>!</h1>
```

We also need a tweak to app/views/layouts/application.html.erb to make this work:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Myapp</title>
    <%= csrf_meta_tags %>

    <%= stylesheet_link_tag 'application',
                            media: 'all',
                           'data-turbolinks-track': 'reload' %>

    <%= javascript_include_tag 'application',
                               'data-turbolinks-track': 'reload' %>
➤   <%= yield :head %>
  </head>
```